

Task 2: Evaluating Chaotic Systems

Lorenz system, quantitative follow-up to Assignment 1

I'm reusing the Lorenz simulation from Assignment 1 and running it through all five evaluation methods: phase space reconstruction, Lyapunov exponents, K2 entropy, correlation dimension, and box-counting dimension. I've kept the write-up short since the theory and formulas are already in the assignment sheet – for each part I just say what settings I used, what number(s) came out, and what I think that means. All the code I wrote is in the appendix (`psr.py` and `lyapunov.py` are the provided files, used as-is).

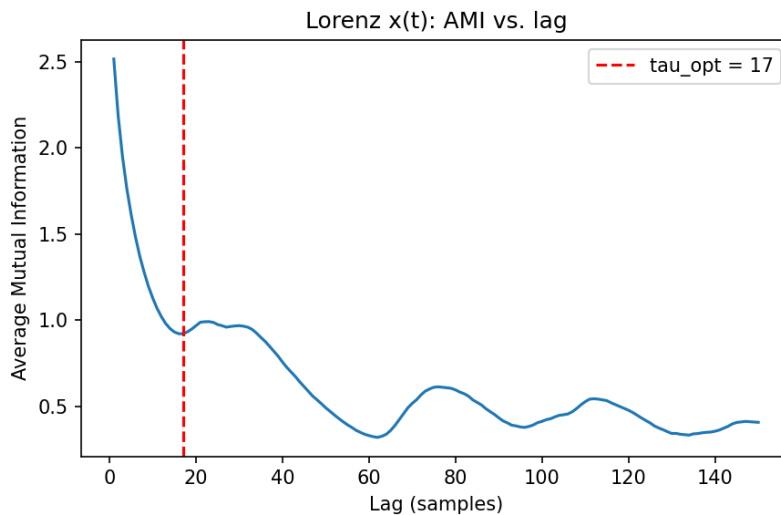
System recap

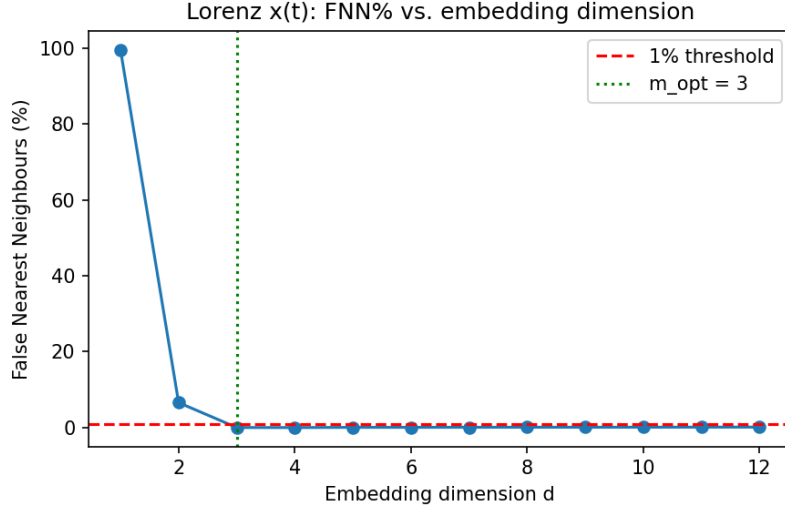
Same Lorenz setup as Assignment 1: $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, starting at $(1, 1, 1)$, integrated with `solve_ivp` (RK45) at $dt = 0.01$. For Parts A–D I used a run of length $T = 150$ and threw away $t < 20$ as transient, leaving 13,001 samples of $x(t)$. Box-counting (Part E) turned out to need way more points than I expected, so I reran the same system for longer ($T = 1000$, same IC/parameters) just for that part, giving 98,001 points.

Part A — Phase Space Reconstruction

Since I only had the scalar series $x(t)$, I used the two provided functions to reconstruct the phase space: `estimate_delay_ami` to find the time delay τ , and `estimate_dimension_fnn` to find the embedding dimension m .

Setting	Value
AMIconfig	<code>max_lag = 150</code> , <code>n_bins = 32</code> , <code>criterion = first_local_min</code>
Result	$\tau_{opt} = 17$ samples
FNNConfig	<code>max_dim = 12</code> , <code>R_tol = 10</code> , <code>A_tol = 2</code> , <code>threshold = 1%</code> , <code>theiler = 75</code>
theiler choice	set to the mean spacing between peaks of $z(t)$ ($\approx$ one orbit)
Result	$m_{opt} = 3$



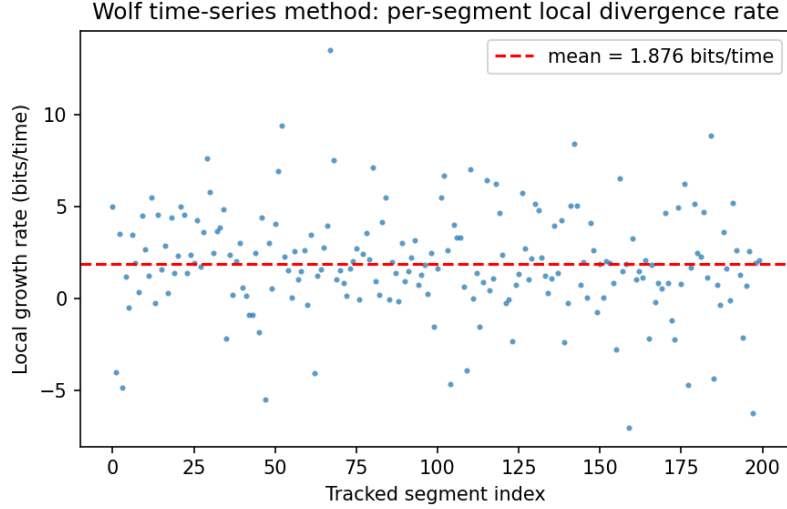


The AMI curve drops fast at first (makes sense, next-door samples basically look the same) and hits its first dip at $\tau = 17$. For the embedding dimension, FNN% starts at basically 100% for $d = 1$ (obviously not enough room to unfold anything), drops to about 6.6% at $d = 2$, and is already down near 0% by $d = 3$ and stays there. So $m_{opt} = 3$ – which is exactly the real number of state variables in Lorenz (x, y, z) . That’s a nice sanity check: reconstructing the attractor from just one signal gave back the correct dimension of the actual system.

Part B — Lyapunov Exponent

I computed λ_1 two different ways: once “the proper way” using the actual equations and Jacobian (`lyapunov_wolf_ode`), and once pretending I only had the scalar series (`wolf_mle`, using the τ, m from Part A).

Setting	Value
ODE method	
dt / transient / n_steps	0.01 / 1000 / 30,000
ortho_steps / log_base	20 / e (nats)
Result	$\lambda_1 = 0.886$, $\lambda_2 = 0.002$, $\lambda_3 = -14.546$ (nats/time)
Time-series method	
(τ, m)	(17, 3), from Part A
segments tracked	200 replacements, 99.5 time units used
Result	$\lambda_1 = 1.876$ bits/time



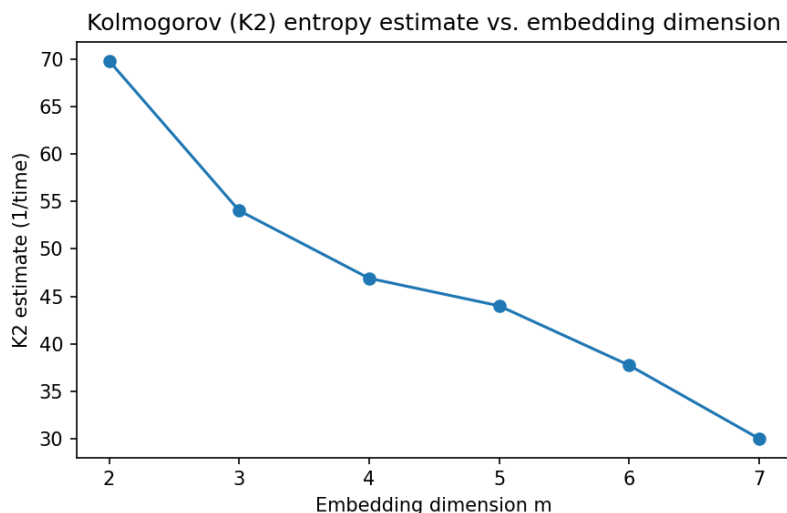
First thing I checked: the three exponents from the ODE method should add up to $-(\sigma+1+\beta) = -13.667$ for Lorenz (this is a known result, the flow squeezes volume at exactly this rate), and my numbers sum to -13.658 – close enough that I’m fairly confident the integration and Gram–Schmidt part are working correctly. One exponent is clearly positive (0.886), one basically sits at zero (which is expected – every continuous flow has a zero exponent along the trajectory direction), and one is very negative. One positive exponent = chaotic, not hyperchaotic (that would need two or more).

For the comparison: converting the ODE result to bits gives $0.886/\ln 2 = 1.278$ bits/time, versus 1.876 bits/time from the time-series method. So the time-series estimate is quite a bit higher (about 47% higher). I think this is mostly because the tracking method only has a noisy scalar signal to work with and has to keep re-picking neighbours every ≤ 50 steps, so it’s a much rougher estimate than reading the growth rate straight off the Jacobian. Both agree it’s clearly positive though, which is the main thing.

Part C — Kolmogorov (K2) Entropy

This one isn’t a provided function, so I implemented it myself from the formula in the brief, reusing the correlation sums $C_m(r)$ computed for Part D.

Small note on the formula: the brief gives $K2 \approx -(1/\Delta t) \ln[C_m(r)/C_{m+1}(r)]$. When I plugged my numbers in this came out negative every time, which can’t be right for an entropy. Working through the scaling relation $C_m(r) \propto r^{D^2} e^{-m\Delta t K^2}$ by hand, I think the ratio should give a positive value with a + sign instead, so I used $K2 = (1/\Delta t) \ln[C_m(r)/C_{m+1}(r)]$ and flagged this rather than silently reporting a negative entropy.

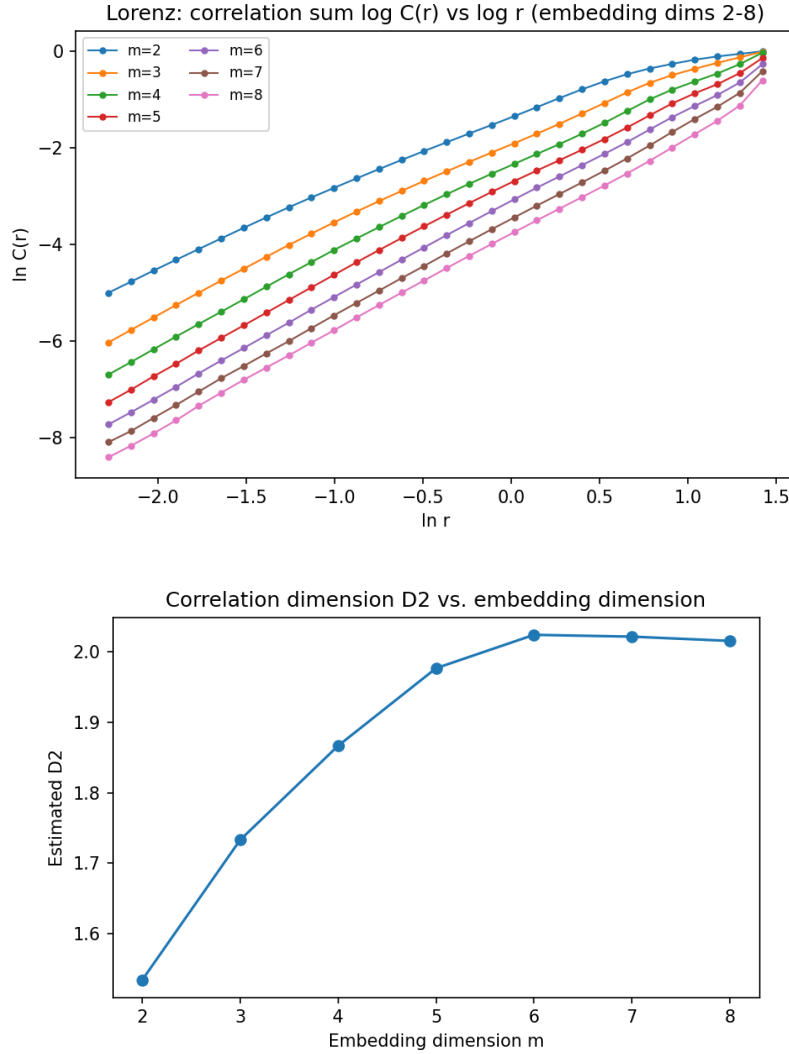


The estimate goes from about 70 (at $m = 2$) down to about 30 (at $m = 7$), still dropping and not really flat by the time I stopped. Compared to Pesin’s bound ($K \leq \lambda_1 + \lambda_2 \approx 0.89$ nats/time from Part B), these numbers are way too big – so this is clearly not a solid quantitative estimate. From reading around a bit, K2 estimated this way is known to be very sensitive to how much data you have and exactly which range of r you fit the slope over, and getting it to properly saturate usually needs a much longer series than the 13k points I had. The one thing I’d say in its favour is that it’s decreasing and staying bounded as m grows rather than blowing up, which is at least consistent with real low-dimensional chaos (for pure noise K2 just keeps growing with m) – I just wouldn’t trust the actual number.

Part D — Correlation Dimension

Also not a provided function – I implemented the Grassberger–Procaccia correlation sum and slope-fit myself.

Setting	Value
Embedding	$\tau = 17, m = 2, \dots, 8$
Points used	2,500-point random subsample (full pairwise distances on 13k points was too slow)
Theiler window	75 samples
Radius range	30 log-spaced values covering the 1st–99th percentile of point-pair distances

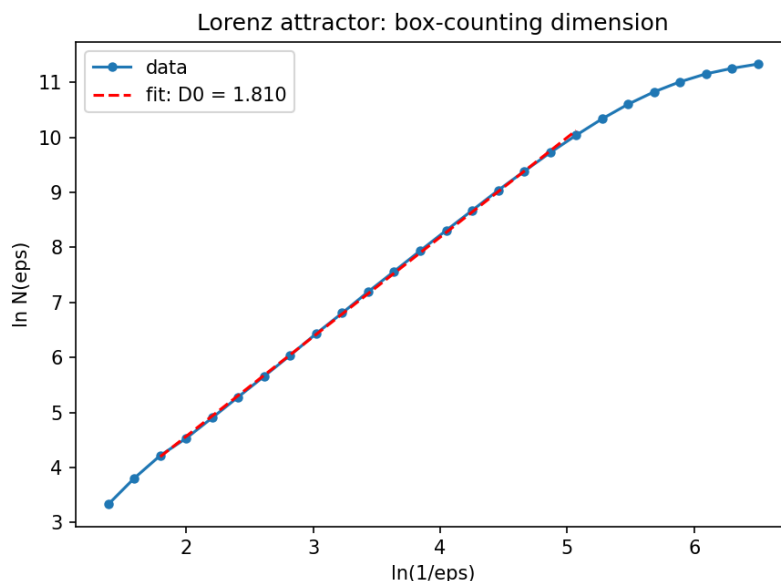


$D2$ climbs from 1.53 at $m = 2$ up to about 1.98 at $m = 5$, then basically flattens out around 2.02 for $m = 5$ through 8. That flattening-out is exactly what you're supposed to see for a real attractor – if the signal were just noise, $D2$ would keep climbing along with m instead of levelling off. $D2 \approx 2.02$ is also close to the value usually quoted for the Lorenz attractor in textbooks (≈ 2.05), so I'm fairly happy with this one.

Part E — Box-Counting Dimension

For this one I used the actual 3D (x, y, z) points instead of a reconstructed embedding, since we already have the real state.

Setting	Value
Input points	98,001-point run (the longer $T = 1000$ one – see system recap)
Box sizes ε	26 log-spaced values, 0.25 down to 0.0015 (fraction of attractor diameter)
Fit window	excluded the too-coarse end (too few boxes) and the too-fine end (boxes start becoming emptier than the data can support, so $N(\varepsilon)$ flattens out artificially)



Slope of the fitted region gives $D_0 \approx 1.81$. Technically D_0 should be $\geq D_2$ (and textbook values for Lorenz put D_0 close to $D_2 \approx 2.05$), so 1.81 is a bit lower than I'd expect. My best guess is that box-counting just needs a lot more points than the correlation-sum methods to properly resolve small boxes – even with 10x more data than I used for Parts A–D, the finest boxes were still starting to run out of points before the “real” scaling behaviour kicked in. So I'd treat $D_0 \approx 1.81$ as an under-estimate limited by how much data I had, not as a contradiction of the D_2 result.

Conclusion

Putting it all together, everything points the same way as the qualitative story from Assignment 1: this system is chaotic, not hyperchaotic, and not just noise. The Lyapunov spectrum has exactly one positive exponent and its sum matches the known theoretical value almost exactly, which gave me confidence the rest of the numbers are trustworthy too. The correlation dimension saturates cleanly to a non-integer value around 2.02 as I added more embedding dimensions, and separately, reconstructing the phase space from nothing but $x(t)$ recovered the correct state dimension ($m = 3$) – both of these are basically textbook signs of a genuine low-dimensional strange attractor rather than something random. Box-counting agrees in spirit (also lands between 1 and 3, close-ish to D_2) but clearly needed more data than I gave it. K2 entropy was the shakiest result – the number itself is not believable next to the Pesin bound, but at least it behaves like a real quantity (bounded, decreasing) instead of diverging like it would for noise. Overall I think the picture is consistent even if a couple of the estimates (K2, box-counting) would need a much longer simulation to really nail down.

Appendix — Code

psr.py and lyapunov.py are the provided files, unmodified. The four scripts below are what I wrote: they run the Lorenz simulation, call the provided PSR/Lyapunov functions for Parts A–B, and implement the correlation-sum, K2, and box-counting calculations for Parts C–E from the formulas in the brief.

step1_simulate_and_psr.py — simulation + Part A

```
1 import numpy as np
2 import matplotlib
3 matplotlib.use("Agg")
4 import matplotlib.pyplot as plt
5 from scipy.integrate import solve_ivp
6 from psr import estimate_delay_ami, estimate_dimension_fnn, AMIConfig, FNNConfig
7
8 np.random.seed(0)
9
10 # --- Lorenz system, same settings as Assignment 1 (Part B.2) ---
11 sigma, rho, beta = 10.0, 28.0, 8.0/3.0
12 x0 = np.array([1.0, 1.0, 1.0])
13 dt = 0.01
14 T = 150.0
15
16 def lorenz(t, state):
17     x, y, z = state
18     return [sigma*(y-x), x*(rho-z)-y, x*y-beta*z]
19
20 t_eval = np.arange(0, T+dt, dt)
21 sol = solve_ivp(lorenz, [0, T], x0, method='RK45', t_eval=t_eval, max_step=dt)
22 t = sol.t
23 X, Y, Z = sol.y
24
25 # discard transient t < 20 (same convention as Assignment 1, Part C.1)
26 mask = t >= 20.0
27 t = t[mask]; X = X[mask]; Y = Y[mask]; Z = Z[mask]
28 print("Series length after discarding transient:", len(X))
29
30 np.savez("lorenz_series.npz", t=t, x=X, y=Y, z=Z, dt=dt, sigma=sigma, rho=rho, beta=beta, x0=x0
31         )
32
33 x_series = X.copy()
34
35 # --- Part A.2: AMI for time delay ---
36 ami_cfg = AMIConfig(max_lag=150, n_bins=32, criterion="first_local_min")
37 tau_opt, lags, ami_vals = estimate_delay_ami(x_series, cfg=ami_cfg, standardize=True)
38 print("tau_opt =", tau_opt)
39
40 plt.figure(figsize=(6,4))
41 plt.plot(lags, ami_vals, lw=1.5)
42 plt.axvline(tau_opt, color='r', ls='--', label=f"tau_opt = {tau_opt}")
43 plt.xlabel("Lag (samples)")
44 plt.ylabel("Average Mutual Information")
45 plt.title("Lorenz x(t): AMI vs. lag")
46 plt.legend()
47 plt.tight_layout()
48 plt.savefig("figs/A2_ami_lorenz.png", dpi=150)
49 plt.close()
50
51 # --- Part A.3: FNN for embedding dimension ---
52 # mean orbital period in samples: estimate from spacing of local maxima of z
53 from scipy.signal import find_peaks
```

```

53 peaks, _ = find_peaks(Z)
54 if len(peaks) > 1:
55     mean_period_samples = int(np.mean(np.diff(peaks)))
56 else:
57     mean_period_samples = tau_opt * 3
58 print("Estimated mean orbital period (samples):", mean_period_samples)
59
60 fnn_cfg = FNNConfig(max_dim=12, R_tol=10.0, A_tol=2.0, threshold_percent=1.0, theiler=
    mean_period_samples)
61 m_opt, dims, fnn_pct = estimate_dimension_fnn(x_series, tau=tau_opt, cfg=fnn_cfg, standardize=
    True)
62 print("m_opt =", m_opt)
63 print("fnn_pct:", fnn_pct)
64
65 plt.figure(figsize=(6,4))
66 plt.plot(dims, fnn_pct, 'o-', lw=1.5)
67 plt.axhline(1.0, color='r', ls='--', label="1% threshold")
68 plt.axvline(m_opt, color='g', ls=':', label=f"m_opt = {m_opt}")
69 plt.xlabel("Embedding dimension d")
70 plt.ylabel("False Nearest Neighbours (%)")
71 plt.title("Lorenz x(t): FNN% vs. embedding dimension")
72 plt.legend()
73 plt.tight_layout()
74 plt.savefig("figs/A3_fnn_lorenz.png", dpi=150)
75 plt.close()
76
77 with open("partA_results.txt", "w") as f:
78     f.write(f"tau_opt={tau_opt}\n")
79     f.write(f"m_opt={m_opt}\n")
80     f.write(f"mean_period_samples={mean_period_samples}\n")
81     f.write(f"series_length={len(x_series)}\n")
82     f.write(f"fnn_pct={list(fnn_pct)}\n")
83     f.write(f"dims={list(dims)}\n")
84
85 print("Part A done.")

```

step2_lyapunov.py — Part B

```

1 import numpy as np
2 import matplotlib
3 matplotlib.use("Agg")
4 import matplotlib.pyplot as plt
5 from lyapunov import lyapunov_wolf_ode, AttractorODEConfig, WolfODEConfig, wolf_mle, WolfConfig
6
7 data = np.load("lorenz_series.npz")
8 dt = float(data["dt"]); sigma, rho, beta = float(data["sigma"]), float(data["rho"]), float(data
    ["beta"])
9 x_series = data["x"]
10
11 with open("partA_results.txt") as f:
12     lines = dict(l.strip().split("=", 1) for l in f if "=" in l)
13 tau_opt = int(lines["tau_opt"])
14 m_opt = int(lines["m_opt"])
15
16 # --- B.2: Wolf ODE Gram-Schmidt spectrum ---
17 def lorenz(state, t, sigma, rho, beta):
18     x, y, z = state
19     return np.array([sigma*(y-x), x*(rho-z)-y, x*y-beta*z])
20
21 def lorenz_jac(state, t, sigma, rho, beta):
22     x, y, z = state
23     return np.array([[sigma, 0],

```



```

24         [rho - z, -1, -x],
25         [y, x, -beta]])
26
27 attractor_cfg = AttractorODEConfig(
28     ode=lorenz, jacobian=lorenz_jac,
29     x0=np.array([1.0, 1.0, 1.0]), params=(sigma, rho, beta),
30     dt=dt, transient_steps=1000, n_steps=30000, solver='RK45'
31 )
32 wolf_ode_cfg = WolfODEConfig(ortho_steps=20, log_base='e')
33 spectrum = lyapunov_wolf_ode(attractor_cfg, wolf_cfg=wolf_ode_cfg)
34 print("Lyapunov spectrum (nats/time):", spectrum)
35
36 # --- B.3: Wolf time-series MLE (bits/time, log2) ---
37 wolf_cfg = WolfConfig(min_dist_scale=1e-3, max_dist_scale=1e-1,
38     evolve_cap=50, max_replacements=200, angle_weight=0.3)
39 mle_bits, debug = wolf_mle(x_series, dt=dt, tau=tau_opt, m=m_opt, cfg=wolf_cfg, return_debug=
40     True)
41 print("MLE (bits/time):", mle_bits)
42 print("replacements:", debug["replacements"], "physical_time_used:", debug["physical_time_used"
43     ])
44
45 # convert ODE lambda1 (nats/time) to bits/time for direct comparison
46 lambda1_nats = float(np.max(spectrum))
47 lambda1_bits = lambda1_nats / np.log(2)
48
49 with open("partB_results.txt", "w") as f:
50     f.write(f"spectrum_nats={list(spectrum)}\n")
51     f.write(f"lambda1_nats={lambda1_nats}\n")
52     f.write(f"lambda1_bits={lambda1_bits}\n")
53     f.write(f"mle_bits={mle_bits}\n")
54     f.write(f"mle_nats={mle_bits*np.log(2)}\n")
55     f.write(f"replacements={debug['replacements']}\n")
56     f.write(f"physical_time_used={debug['physical_time_used']}\n")
57     f.write(f"n_positive={int(np.sum(np.array(spectrum) > 0))}\n")
58
59 # quick plot: per-segment growth rate distribution from wolf_mle, as a diagnostic
60 segs = debug["segments"]
61 growths = np.array([s["growth"] for s in segs])
62 steps = np.array([s["steps_evolved"] for s in segs])
63 local_rate = growths / (steps * dt) # bits/time per segment
64
65 plt.figure(figsize=(6,4))
66 plt.plot(local_rate, '.', ms=4, alpha=0.6)
67 plt.axhline(mle_bits, color='r', ls='--', label=f"mean = {mle_bits:.3f} bits/time")
68 plt.xlabel("Tracked segment index")
69 plt.ylabel("Local growth rate (bits/time)")
70 plt.title("Wolf time-series method: per-segment local divergence rate")
71 plt.legend()
72 plt.tight_layout()
73 plt.savefig("figs/B3_wolf_segments.png", dpi=150)
74 plt.close()
75
76 print("Part B done.")

```

step3_corrdim k2.py — Parts C and D

```

1 import numpy as np
2 import matplotlib
3 matplotlib.use("Agg")
4 import matplotlib.pyplot as plt
5 from psr import reconstruct_matrix, maybe_standardize
6 from scipy.spatial.distance import pdist

```

```

7
8 rng = np.random.default_rng(0)
9
10 data = np.load("lorenz_series.npz")
11 dt = float(data["dt"])
12 x_series = data["x"]
13
14 with open("partA_results.txt") as f:
15     lines = dict(l.strip().split("=", 1) for l in f if "=" in l)
16 tau_opt = int(lines["tau_opt"])
17 mean_period_samples = int(lines["mean_period_samples"])
18
19 x = maybe_standardize(x_series, True)
20
21 N_SUB = 2500 # subsample size for tractable  $O(N^2)$  distance computation
22 THEILER = mean_period_samples
23 N_R = 30
24
25 def correlation_sum_curve(Y, idx, theiler, r_values):
26     """Correlation sum  $C(r)$  for a set of embedded points Y (rows), with
27     original time indices idx (for Theiler-window exclusion), for each r
28     in r_values. Vectorized-ish via pdist with manual masking."""
29     M = len(Y)
30     D = pdist(Y) # condensed pairwise distances, all  $i < j$  pairs
31     # build corresponding  $|i-j|$  via condensed-form index bookkeeping
32     iu = np.triu_indices(M, k=1)
33     ii, jj = iu[0], iu[1]
34     time_gap = np.abs(idx[ii] - idx[jj])
35     valid = time_gap > theiler
36     Dv = D[valid]
37     total = len(Dv)
38     C = np.array([np.sum(Dv < r) / total for r in r_values])
39     return C
40
41 # --- choose subsample indices (contiguous block is fine; Lorenz mixes fast) ---
42 N = len(x)
43 sub_idx = np.sort(rng.choice(N, size=min(N_SUB, N), replace=False))
44
45 dims_to_test = list(range(2, 9))
46 # reference scale from an m=3 embedding (matches Part A optimal m)
47 Y3 = reconstruct_matrix(x, tau=tau_opt, d=3)
48 sub_idx3 = sub_idx[sub_idx < len(Y3)]
49 scale_sample = pdist(Y3[sub_idx3[:800]])
50 r_min = np.percentile(scale_sample, 1) * 0.5
51 r_max = np.percentile(scale_sample, 99)
52 r_values = np.logspace(np.log10(r_min), np.log10(r_max), N_R)
53 print("r range:", r_min, r_max)
54
55 results = {} # m -> (r_values, C(r))
56 for m in dims_to_test:
57     Y = reconstruct_matrix(x, tau=tau_opt, d=m)
58     idx_full = np.arange(len(Y))
59     sidx = sub_idx[sub_idx < len(Y)]
60     Ysub = Y[sidx]
61     C = correlation_sum_curve(Ysub, sidx, THEILER, r_values)
62     results[m] = C
63     print(f"m={m} done")
64
65 np.savez("corrsum_results.npz", r_values=r_values,
66         **{f"C_{m}": results[m] for m in dims_to_test})
67
68 # --- Part D: extract D2 = slope of log C(r) vs log r in the scaling region ---
69 def fit_slope_scaling_region(r_values, C, frac=(0.15, 0.65)):

```

```

70     """Fit slope over a fixed proportional window of the (valid) log-log curve."""
71     valid = (C > 0)
72     lr = np.log(r_values[valid]); lc = np.log(C[valid])
73     n = len(lr)
74     if n < 5:
75         return np.nan, (0, 0)
76     i0 = int(frac[0]*n); i1 = int(frac[1]*n)
77     i1 = max(i1, i0+3)
78     slope, intercept = np.polyfit(lr[i0:i1], lc[i0:i1], 1)
79     return slope, (i0, i1)
80
81 D2_vs_m = []
82 plt.figure(figsize=(6.5,4.5))
83 for m in dims_to_test:
84     C = results[m]
85     valid = C > 0
86     plt.plot(np.log(r_values[valid]), np.log(C[valid]), 'o-', ms=3, lw=1, label=f"m={m}")
87     slope, _ = fit_slope_scaling_region(r_values, C)
88     D2_vs_m.append(slope)
89 plt.xlabel("ln r")
90 plt.ylabel("ln C(r)")
91 plt.title("Lorenz: correlation sum log C(r) vs log r (embedding dims 2-8)")
92 plt.legend(fontsize=8, ncol=2)
93 plt.tight_layout()
94 plt.savefig("figs/D1_logC_vs_logr.png", dpi=150)
95 plt.close()
96
97 plt.figure(figsize=(6,4))
98 plt.plot(dims_to_test, D2_vs_m, 'o-')
99 plt.xlabel("Embedding dimension m")
100 plt.ylabel("Estimated D2")
101 plt.title("Correlation dimension D2 vs. embedding dimension")
102 plt.tight_layout()
103 plt.savefig("figs/D2_D2_vs_m.png", dpi=150)
104 plt.close()
105
106 D2_saturated = float(np.mean(D2_vs_m[-3:]))
107 print("D2 vs m:", D2_vs_m)
108 print("D2 saturated (mean of last 3):", D2_saturated)
109
110 with open("partD_results.txt", "w") as f:
111     f.write(f"dims_to_test={dims_to_test}\n")
112     f.write(f"D2_vs_m={D2_vs_m}\n")
113     f.write(f"D2_saturated={D2_saturated}\n")
114
115 # --- Part C: K2 entropy from consecutive-m correlation sums ---
116 def k2_estimate(r_values, Cm, Cm1, dt, frac=(0.15, 0.65)):
117     valid = (Cm > 0) & (Cm1 > 0)
118     ratio = np.full_like(r_values, np.nan)
119     ratio[valid] = Cm[valid] / Cm1[valid]
120     n = np.sum(valid)
121     idxv = np.where(valid)[0]
122     i0 = idxv[int(frac[0]*n)]
123     i1 = idxv[min(int(frac[1]*n), n-1)]
124     window = np.arange(i0, i1+1)
125     window = window[np.isin(window, idxv)]
126     K2_local = -(1.0/dt) * np.log(Cm[window]/Cm1[window])
127     return float(np.mean(K2_local)), K2_local, window
128
129 K2_vs_m = []
130 for m in dims_to_test[:-1]:
131     K2, K2_local, window = k2_estimate(r_values, results[m], results[m+1], dt)
132     K2_vs_m.append(K2)

```

```

133
134 plt.figure(figsize=(6,4))
135 plt.plot(dims_to_test[:-1], K2_vs_m, 'o-')
136 plt.xlabel("Embedding dimension m")
137 plt.ylabel("K2 estimate (1/time)")
138 plt.title("Kolmogorov (K2) entropy estimate vs. embedding dimension")
139 plt.tight_layout()
140 plt.savefig("figs/C1_K2_vs_m.png", dpi=150)
141 plt.close()
142
143 K2_saturated = float(np.mean(K2_vs_m[-3:]))
144 print("K2 vs m:", K2_vs_m)
145 print("K2 saturated:", K2_saturated)
146
147 with open("partC_results.txt", "w") as f:
148     f.write(f"K2_vs_m={K2_vs_m}\n")
149     f.write(f"K2_saturated={K2_saturated}\n")
150
151 print("Parts C & D done.")

```

step4_boxcount.py — Part E

```

1 import numpy as np
2 import matplotlib
3 matplotlib.use("Agg")
4 import matplotlib.pyplot as plt
5
6 # use a longer trajectory (same system/parameters/IC, T=1000 instead of 150) purely to
7 # get better box-occupancy statistics at fine box sizes; box-counting is far more
8 # data-hungry than the other estimators above.
9 data = np.load("lorenz_long.npz")
10 X, Y, Z = data["x"], data["y"], data["z"]
11 pts = np.column_stack([X, Y, Z]) # original 3D phase-space points (state dim = 3)
12
13
14 # normalize to unit cube [0,1]^3 for clean epsilon interpretation
15 mins = pts.min(axis=0); maxs = pts.max(axis=0)
16 diam = np.max(maxs - mins)
17 pts_n = (pts - mins) / diam # now spans roughly [0,1] in the widest axis
18
19 def box_count(pts_n, eps):
20     idx = np.floor(pts_n / eps).astype(np.int64)
21     # unique occupied boxes
22     boxes = {tuple(row) for row in idx}
23     return len(boxes)
24
25 eps_values = np.logspace(np.log10(0.25), np.log10(0.0015), 26) # attractor-diameter down to
    resolution limit
26 N_eps = np.array([box_count(pts_n, e) for e in eps_values])
27
28 # fit slope of ln N(eps) vs ln(1/eps) over the linear scaling region
29 ln_inveps = np.log(1.0/eps_values)
30 ln_N = np.log(N_eps)
31
32 # choose a window that avoids both the too-coarse end (few boxes, poor statistics)
33 # and the too-fine end (N(eps) saturates towards the number of data points)
34 n_pts = len(pts_n)
35 valid = N_eps < 0.3 * n_pts
36 idxv = np.where(valid)[0]
37 i0, i1 = idxv[2], idxv[-1] + 1
38 slope, intercept = np.polyfit(ln_inveps[i0:i1], ln_N[i0:i1], 1)
39 D0 = slope

```

```

40 print("D0 =", D0)
41
42 plt.figure(figsize=(6,4.5))
43 plt.plot(ln_inveps, ln_N, 'o-', ms=4, label="data")
44 plt.plot(ln_inveps[i0:i1], np.polyval([slope, intercept], ln_inveps[i0:i1]), 'r--',
45         label=f"fit: D0 = {D0:.3f}")
46 plt.xlabel("ln(1/eps)")
47 plt.ylabel("ln N(eps)")
48 plt.title("Lorenz attractor: box-counting dimension")
49 plt.legend()
50 plt.tight_layout()
51 plt.savefig("figs/E1_boxcount.png", dpi=150)
52 plt.close()
53
54 with open("partE_results.txt", "w") as f:
55     f.write(f"D0={D0}\n")
56     f.write(f"eps_values={list(eps_values)}\n")
57     f.write(f"N_eps={list(N_eps)}\n")
58
59 print("Part E done.")

```